

Análisis Técnico y Arquitectura de Software: Implementación de PSO para Cloud Scheduling

1. Introducción al código

El presente código fuente constituye una solución algorítmica robusta para el problema de asignación de tareas en infraestructuras de computación en la nube (*Cloud Scheduling*). El objetivo principal del programa es distribuir un lote de peticiones computacionales heterogéneas entre un clúster de servidores, buscando minimizar el **Makespan** (el tiempo total de finalización del servidor más saturado).

Dado que este problema pertenece a la clase NP-duro, los enfoques deterministas tradicionales sufren de explosión combinatoria al aumentar el número de tareas. Por lo tanto, el código implementa una metaheurística fundamentada en Inteligencia Artificial, específicamente en algoritmos bioinspirados. Se utiliza la **Optimización por Enjambre de Partículas (PSO - Particle Swarm Optimization)**, un enfoque de Inteligencia Colectiva (*Swarm Intelligence*) que emula matemáticamente el comportamiento social y el intercambio de información observado en bandadas de aves o bancos de peces. El código traduce estas interacciones biológicas en operaciones vectoriales para navegar eficientemente por un espacio de búsqueda multidimensional y encontrar asignaciones de carga óptimas o casi óptimas en tiempos de ejecución viables.

2. Librerías utilizadas

El desarrollo del proyecto se apoya en módulos nativos y de terceros del ecosistema de Python, seleccionados estratégicamente para garantizar un rendimiento óptimo:

- **NumPy ([numpy](#)):** Es el motor matemático central del algoritmo. Se utiliza para representar las partículas, sus posiciones y velocidades mediante arreglos multidimensionales ([ndarrays](#)). NumPy aporta la vectorización de operaciones matemáticas, lo que permite actualizar el estado de todo el enjambre simultáneamente sin depender de bucles [for](#) nativos anidados, reduciendo drásticamente la complejidad temporal y el *overhead* computacional.
- **Matplotlib ([matplotlib.pyplot](#)):** Librería estándar para la generación de gráficos en 2D. En este proyecto, se implementa para trazar la curva de convergencia del algoritmo, graficando la evolución de la función de aptitud a lo largo de las iteraciones. Provee una validación visual indispensable en reportes académicos.
- **CSV ([csv](#)):** Módulo nativo empleado para la lectura e ingesta de datos. Permite desacoplar los parámetros del entorno (las cargas de trabajo) del código fuente, leyendo las tareas estructuradas (ID, peso, descripción) desde archivos externos, lo que dota al sistema de flexibilidad para simular diferentes escenarios de estrés.
- **JSON ([json](#)):** Módulo nativo utilizado para la serialización de datos. Tras alcanzar la convergencia, la solución óptima es exportada en formato JSON, emulando la arquitectura de microservicios y orquestadores modernos (como Kubernetes), facilitando la auditoría y la interoperabilidad con otras plataformas.

- **OS (os)**: Se utiliza para la gestión de rutas y validación de existencia de archivos en el sistema operativo, garantizando que el entorno de experimentación cuente con los datos base antes de iniciar la optimización.

3. Lectura y procesamiento de datos

El flujo de datos inicia con la ingesta dinámica de las cargas de trabajo desde el entorno local. La función `cargar_tareas_desde_csv` procesa un archivo con extensión `.csv` que simula una cola de peticiones en un servidor web o entorno IoT.

El archivo contiene el peso computacional de cada tarea (medido heurísticamente en millones de instrucciones o milisegundos base). Durante la lectura, el programa itera sobre los registros utilizando un `DictReader` y extrae los pesos, almacenándolos en un arreglo vectorizado de NumPy denominado `PESO_TAREAS`. Esta etapa de preprocesamiento es crítica, ya que tener los datos en memoria contigua en forma de un `array` de NumPy garantiza que el algoritmo PSO, que evaluará la función de aptitud miles de veces, acceda a los pesos con un tiempo de complejidad $O(1)$, evitando cuellos de botella de I/O (Entrada/Salida) durante la fase de optimización.

4. Representación de partículas

En el contexto del algoritmo PSO, una "partícula" representa una solución candidata al problema. En este código, la representación del *Scheduling* se aborda mediante una codificación continua.

La posición de una partícula se define como un vector $X \in \mathbb{R}^D$, donde la dimensionalidad D es exactamente igual al número de tareas a procesar. Cada índice del vector corresponde a una tarea específica, y el valor flotante almacenado en dicho índice representa el servidor al que está asignada.

Dado que el PSO es un optimizador de dominio continuo, pero el problema de asignación es discreto (no existe el "servidor 1.5"), el código utiliza un mapeo mediante redondeo matemático. Posteriormente, durante la evaluación, un valor como `1.74` se redondea al entero `2` y se delimita mediante la función `np.clip` para asegurar que el índice resultante sea válido dentro del rango de servidores disponibles (por ejemplo, del 0 al 2).

5. Inicialización del PSO

La clase `OptimizadoPSO` actúa como el orquestador del enjambre. Durante el proceso de instanciación, se crea un arreglo de objetos de la clase `Particula`.

La inicialización del espacio de búsqueda es estrictamente estocástica. Las posiciones iniciales se generan utilizando distribuciones uniformes (`np.random.uniform`) acotadas por los límites del problema (el servidor ID 0 como mínimo y el ID $N-1$ como máximo). Del mismo modo, las velocidades iniciales se configuran aleatoriamente para garantizar que las partículas comiencen a explorar el espacio de soluciones en diversas direcciones y no se concentren prematuramente en un subespacio específico.

En esta etapa, cada partícula guarda su configuración inicial como su "mejor posición histórica individual" (`pBest`), con un *fitness* evaluado en infinito (`float('inf')`), garantizando que

la primera iteración registre inmediatamente una mejora. Asimismo, se inicializa la variable global **gBest**, la cual será el faro de convergencia de todo el sistema.

6. Función de fitness

La función de aptitud o evaluación, denominada **evaluar_balanceo_carga**, es el núcleo matemático que dicta la calidad de las soluciones. Recibe el vector continuo de una partícula y realiza el mapeo al dominio discreto.

Posteriormente, itera sobre las tareas asignadas y acumula los tiempos de procesamiento en un arreglo auxiliar llamado **tiempo_servidores**. El cálculo del tiempo real considera la heterogeneidad de la infraestructura, multiplicando el peso de la tarea por el factor de capacidad computacional del servidor destino.

Finalmente, la función retorna el **Makespan**, obtenido directamente mediante **np.max(tiempo_servidores)**. El paradigma del algoritmo es de minimización: al buscar el valor más bajo posible para el máximo de los tiempos, el enjambre se ve forzado matemáticamente a redistribuir las cargas pesadas de manera equitativa, logrando un balanceo de recursos óptimo y evitando la saturación térmica o de procesamiento en nodos individuales.

7. Actualización de partículas

La evolución geométrica de las soluciones se rige por las dos ecuaciones clásicas de la física del enjambre. En el ciclo principal del código, cada iteración actualiza el vector de velocidad y la posición mediante las siguientes componentes:

1. **Influencia de Inercia ($w \cdot v$):** El parámetro w (típicamente 0.7) permite a la partícula mantener su trayectoria anterior, regulando la amplitud de la exploración global y evitando fluctuaciones caóticas extremas.
2. **Influencia Cognitiva ($c_1 \cdot r_1 \cdot (pBest - x)$):** Actúa como la memoria local del individuo. El algoritmo atrae a la partícula hacia el punto del espacio donde ella misma obtuvo su mejor *Makespan* en el pasado.
3. **Influencia Social ($c_2 \cdot r_2 \cdot (gBest - x)$):** Actúa como el intercambio de información colectiva. La partícula es traccionada hacia la mejor solución encontrada por cualquier integrante del enjambre hasta ese momento.

Los componentes r_1 y r_2 introducen vectores de variables aleatorias uniformes que otorgan la estocasticidad necesaria para evitar quedar atrapados en óptimos locales. Finalmente, la nueva velocidad se suma a la posición actual ($X^{t+1} = X^t + V^{t+1}$), desplazando a la solución en el espacio multidimensional hacia regiones más prometedoras.

8. Iteraciones y convergencia

El comportamiento dinámico del algoritmo evidencia la transición clásica de exploración a explotación. En las simulaciones reales proporcionadas por el código, el sistema inicia evaluando posiciones aleatorias con altos grados de ineficiencia.

En un caso de prueba típico, se observa que en las iteraciones iniciales (fase de exploración), el *Makespan* arranca en valores deficientes cercanos a los **170 ms**. A medida que las partículas comparten información sobre los gradientes de aptitud a través del componente social (\$gBest\$), el enjambre descarta drásticamente configuraciones erróneas. Para la iteración número 8, las fuerzas de atracción del sistema fuerzan una alineación colectiva hacia el valle del óptimo global, provocando una caída brusca en el error hasta alcanzar un valor de **165 ms**.

A partir de este punto, el algoritmo se estabiliza. La curva plana posterior indica que las partículas han entrado en la fase de convergencia o explotación fina, orbitando muy cerca del óptimo para intentar refinar fracciones de milisegundo, confirmando matemáticamente que el enjambre ha hallado una solución altamente competitiva para el lote de tareas propuesto.

9. Generación de gráficas

El almacenamiento del rendimiento iterativo se gestiona a través de la lista *historial_scores*. En cada ciclo de la optimización, se inserta el valor del *Makespan* global (*gBest*).

Finalizado el ciclo de cálculo, se invoca a Matplotlib para la visualización de resultados. El eje de las abscisas (X) cuantifica el paso del tiempo discreto en iteraciones (épocas), mientras que el eje de las ordenadas (Y) mide el *Makespan* en milisegundos bajo una política heurística donde "menor es mejor". La inclusión de anotaciones programáticas y marcadores en la gráfica proporciona una prueba concluyente del rendimiento del algoritmo, facilitando la comprensión fenomenológica de la tasa de aprendizaje del modelo y el punto exacto de convergencia.

10. Flujo completo del programa

La arquitectura del script se rige bajo un modelo procedural-orientado a objetos invocado desde el bloque condicional principal `if __name__ == "__main__":`. El flujo de ejecución es determinista y secuencial:

1. Se validan y aprovisionan los datos iniciales escribiendo y leyendo el archivo CSV respectivo.
2. Se dimensionan los vectores estructurales, parametrizando los límites continuos del espacio algorítmico basados en la infraestructura disponible (\$0\$ a \$N-1\$ servidores).
3. Se instancia el objeto *OptimizadoPSO* pasando la función de evaluación, las dimensiones de la matriz y los hiperparámetros del motor cognitivo.
4. Se desencadena el método *.ejecutar()*, el cual realiza las iteraciones limitadas en tiempo de CPU.
5. Se realiza la traducción de la solución de punto flotante a su forma final entera discreta y se notifica vía salida estándar (consola).
6. Se exportan los *logs* con los mapeos precisos de tareas-servidor hacia un archivo JSON.
7. Se renderiza e interrumpe el hilo principal para mostrar la interfaz gráfica de Matplotlib.

11. Análisis técnico del código

Desde una perspectiva de la Ingeniería de Software y la Teoría de la Computación, la implementación destaca por su eficiencia. La complejidad temporal del bucle principal es aproximadamente de $O(I \times P \times D)$, donde I son las iteraciones máximas, P el número de partículas en el enjambre y D la dimensionalidad de las tareas.

Al aprovechar el motor subyacente en C de NumPy para las sumas y multiplicaciones vectoriales de las ecuaciones cinemáticas, el factor constante de la complejidad es marginal. Esto representa una ventaja colosal frente a enfoques iterativos puramente construidos en Python o frente a algoritmos de búsqueda exhaustiva factorial $O(N!)$.

Como área de mejora técnica, el sistema podría paralelizar la evaluación de la función *fitness* empleando el módulo `multiprocessing`, dado que las partículas en una misma iteración son matemáticamente independientes hasta el momento de actualizar el valor del `gBest`.

12. Conclusiones técnicas

El proyecto desarrollado demuestra fehacientemente la viabilidad de implementar metaheurísticas estocásticas para optimizar la arquitectura subyacente de la nube. La Optimización por Enjambre de Partículas ha comprobado ser un mecanismo extremadamente ágil y con un bajo costo de implementación paramétrica (a diferencia de operadores genéticos más pesados).

La convergencia exitosa desde escenarios iniciales saturados hacia asignaciones que minimizan equitativamente el *Makespan* refleja cómo las interacciones inspiradas en dinámicas sociales biológicas superan en órdenes de magnitud a los algoritmos empíricos y los planificadores (schedulers) de asignación secuencial o Aleatoria (*Round Robin*), consolidando a los algoritmos bioinspirados como pilares esenciales para el desarrollo de sistemas distribuidos y la gestión térmica-computacional en el mundo real industrial.